INTERNET ARCHIVE WaybackMachine https://wiki.linaro.org/AOSP/Boards/AM65xEVM Go
2 captures
21 Jun 2021
MAY JUN JUL
2020 21 2021 2022
About this capture

Date of last update: 25 Apr 2019

Contents

1. Android Pie with 4.19 on AM65x EVM
    1. Baseline
    2. Preparing the Tools
    3. Obtain and Building the Software
        1. ATF
        2. OP-TEE
        3. SYSFW
        4. U-Boot
        5. Kernel
        6. Android
    4. Flashing and Booting
        1. eMMC Boot
2. Android Oreo with 4.14 on AM65x EVM
    1. Baseline
    2. Preparing the Tools
    3. Hardware Connections
    4. Obtaining and Building the Software
        1. ATF
        2. OP-TEE
        3. SYSFW
        4. U-Boot
        5. Kernel
        6. AFS
    5. Flashing and Booting
        1. UART Boot: Recovery or First time Flash and Boot
        2. SD Boot: Recovery or First time Flash and Boot
        3. eMMC Boot: Production Flash and Boot
    6. Bringing up Ethernet on Android
3. How to build the LCR AM65X Pie build
    1. get the linaro-build.sh script
    2. download all the sources and build images
    3. Prepare the sd card for boot
    4. Configure SYSBOOT DIP-switch for booting from SDCard
    5. Boot to fastboot mode with the SDCard
    6. Flashing the firmware images, and android images
    7. Configure SYSBOOT DIP-switch for booting from eMMC
    8. Boot Android Images
4. References

# Android Pie with 4.19 on AM65x EVM

## Baseline

```
* Android: Android Pie
* Kernel: 4.19.y
* U-Boot: 2019.01
* ATF: 2019.00
* OP-TEE: 2019.00
* SYS-FW: v2019.02
```

## Preparing the Tools

1. Download and install required Linaro toolchains (ARM32 toolchain is needed for Cortex-R5F core, and ARM64 toolchain is needed for Cortex-A53):

```
$ cd ~/Downloads
$ wget https://releases.linaro.org/components/toolchain/binaries/7.3-2018.05/aarch64-linux-gnu/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu.tar.xz
$ wget https://releases.linaro.org/components/toolchain/binaries/7.3-2018.05/arm-eabi/gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi.tar.xz
$ sudo tar xJvf gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu.tar.xz -C /opt/
$ sudo tar xJvf gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi.tar.xz -C /opt
```

2. Download toolchains for building the kernel:

```
$ cd /opt
$ sudo git clone --depth=1 https://android.googlesource.com/platform/prebuilts/clang/host/linux-x86 clang-linux-x86
$ sudo git clone --depth=1 https://android.googlesource.com/platform/prebuilts/gcc/linux-x86/aarch64/aarch64-linux-android-4.9 aarch64-gcc-4.9
```

3. Make OTG USB work without root. Create file `/etc/udev/rules.d/51-android.rules` and add next line to it:

```
SUBSYSTEM=="usb", ATTR{idVendor}=="0451", MODE="0660", GROUP="plugdev"
```

and run next commands:

```
$ sudo udevadm control --reload
$ sudo udevadm trigger
```

4. Make the minicom work without root:

```
$ sudo usermod -a -G dialout $USER
```

then logout/login in your OS.

5. Install packages required for building:

```
$ sudo apt install git ccache minicom make libncurses5-dev libssl-dev bc bison flex mtools device-tree-compiler
$ pip install pycrypto
```

6. Create directory for all repositories:

```
$ mkdir ~/repos
```

We will use this directory to keep the source code for all software. If you'd like to use another location, please replace `~/repos` with desired directory further.

## Obtain and Building the Software

### ATF

Obtain ATF:

```
$ cd ~/repos
$ git clone git://git.ti.com/atf/arm-trusted-firmware.git
$ cd arm-trusted-firmware
$ git checkout ti2019.00
```

Configure shell environment for cross-compiling:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu/bin:$PATH
$ export CROSS_COMPILE="ccache aarch64-linux-gnu-"
$ export ARCH=aarch64
```

Build ATF:

```
$ make PLAT=k3 TARGET_BOARD=generic SPD=opteed
```

### OP-TEE

Obtain OP-TEE:

```
$ cd ~/repos
$ git clone git://git.ti.com/optee/ti-optee-os.git
$ cd ti-optee-os
$ git checkout ti2019.00
```

Configure shell environment:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu/bin:$PATH
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi/bin:$PATH
$ export CROSS_COMPILE64="ccache aarch64-linux-gnu-"
$ export CROSS_COMPILE32="ccache arm-eabi-"
$ unset ARCH
```

Build OP-TEE:

```
$ make PLATFORM=k3-am65x CFG_ARM64_core=y
```

### SYSFW

Obtain SYSFW:

```
$ cd ~/repos
$ git clone git://git.ti.com/processor-firmware/system-firmware-image-gen.git ti-sysfw
$ cd ti-sysfw
$ git checkout 8dc827bac89152ec4185716df7d84abb63824430
```

Configure shell environment:

```
$ export CROSS_COMPILE="ccache arm-eabi-"
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi/bin:$PATH
$ export ARCH=arm
```

Build SysFW:

```
$ make
```

### U-BOOT

Obtain U-Boot:

```
$ cd ~/repos
$ git clone git://git.ti.com/ti-u-boot/ti-u-boot.git
$ cd ti-u-boot
$ git checkout ti-u-boot-2019.01
```

**NOTE**: verified commit is 6844f15d93f3e66dd905a12b69d09d66b17c8888.

Configure shell environment for Cortex-R5F build:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi/bin:$PATH
$ export CROSS_COMPILE="ccache arm-eabi-"
$ export ARCH=arm
$ unset ATF
$ unset TEE
```

Build U-Boot for Cortex-RF5 core:

```
$ make O=/tmp/r5 distclean
$ make O=/tmp/r5 am65x_evm_r5_defconfig
$ make O=/tmp/r5 -j4
```

Configure shell environment for Cortex-A53 build:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu/bin:$PATH
$ export CROSS_COMPILE="ccache aarch64-linux-gnu-"
$ export ARCH=arm
$ export ATF=~/repos/ti-atf/build/k3/generic/release/bl31.bin
$ export TEE=~/repos/ti-optee-os/out/arm-plat-k3/core/tee-pager.bin
```

Build U-Boot for Cortex-A53 core:

```
$ make O=/tmp/a53 distclean
$ make O=/tmp/a53 am65x_evm_a53_defconfig
$ make O=/tmp/a53 -j4
```

### KERNEL

Obtain Android kernel:

```
$ cd ~/repos
$ git clone git://git.ti.com/ti-linux-kernel/ti-linux-kernel.git ti-linux
$ cd ti-linux
$ git checkout ti-android-linux-4.19.y
```

Configure kernel:

```
$ make -j4 ARCH=arm64 distclean
$ ./ti_config_fragments/defconfig_builder.sh -t ti_sdk_arm64_android_release
$ make ARCH=arm64 ti_sdk_arm64_android_release_defconfig
```

Configure the shell environment:

```
$ export PATH=/opt/aarch64-gcc-4.9/bin:$PATH
$ export PATH=/opt/clang-linux-x86/clang-r349610/bin:$PATH
$ export CLANG_TRIPLE=aarch64-linux-gnu-
$ export CROSS_COMPILE=aarch64-linux-android-
$ export ARCH=arm64
```

Build kernel (using clang):

```
$ make -j4 CC="ccache clang" HOSTCC=clang Image dtbs modules
```

### ANDROID

Obtain Android:

```
$ cd ~/repos
$ mkdir ti-android-pie
$ cd ti-android-pie
$ repo init -u git://git.ti.com/android/manifest.git -b pie-core-release
$ repo sync -j20
```

Configure and build Android:

```
$ export KERNELDIR=~/repos/ti-linux
$ . ./build/envsetup.sh
$ lunch am65xevm_full-userdebug
$ make -j4
```

## Flashing and Booting

### EMMC BOOT

Configure SYSBOOT DIP-switch for booting from eMMC (user area mode):

```
SW2[0:3] = 0000
SW3[0:3] = 0110
SW4[0:1] = 00
```

Prepare the U-Boot environment:

```
=> env default -f -a
=> setenv partitions $partitions_android
=> env save
=> fastboot 0
```

Copy images to resulting dir:

```
$ mkdir /tmp/images
$ cd /tmp/images
$ cp ~/repos/ti-android-pie/out/target/product/am65xevm/*.img .
$ cp ~/repos/ti-sysfw/sysfw.itb .
$ cp /tmp/a53/tispl.bin .
$ cp /tmp/a53/u-boot.img .
$ cp /tmp/r5/tiboot3.bin .
```

Create `bootloader` image:

```
$ dd if=/dev/zero of=bootloader.img bs=1M count=8
$ /sbin/mkfs.vfat bootloader.img
$ mcopy -i bootloader.img tispl.bin ::tispl.bin
$ mcopy -i bootloader.img u-boot.img ::u-boot.img
$ mcopy -i bootloader.img sysfw.itb ::sysfw.itb
```

Flash images to eMMC:

```
$ fastboot oem format
$ fastboot flash tiboot3 tiboot3.bin
$ fastboot flash bootloader bootloader.img
$ fastboot flash vendor vendor.img
$ fastboot flash userdata userdata.img
$ fastboot flash boot boot_fit.img
$ fastboot flash system system.img
```

Reboot the board:

```
$ fastboot reboot
```

# Android Oreo with 4.14 on AM65x EVM

This page captures the latest Android baseline details for TI AM65XGPEVM.

Hardware and Supporting Documentation: http://www.ti.com/tool/TMDX654GPEVM

## Baseline

```
* Android: 8.1/ Oreo MR1
* Kernel: 4.14.y
* Uboot: 2018.01
* ATF: 2.0
* OPTEE: 3.4.0
* SysFW: 2018.10
```

## Preparing the Tools

1. Download and install required toolchains (ARM32 toolchain is needed for Cortex-R5F core, and ARM64 toolchain is needed for Cortex-A53):

```
$ cd ~/Downloads
$ wget https://releases.linaro.org/components/toolchain/binaries/7.3-2018.05/aarch64-linux-gnu/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu.tar.xz
$ wget https://releases.linaro.org/components/toolchain/binaries/7.3-2018.05/arm-eabi/gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi.tar.xz
$ sudo tar xJvf gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu.tar.xz -C /opt/
$ sudo tar xJvf gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi.tar.xz -C /opt
```

2. Make OTG USB work without root. Create file `/etc/udev/rules.d/51-android.rules` and add next line to it:

```
SUBSYSTEM=="usb", ATTR{idVendor}=="0451", MODE="0660", GROUP="plugdev"
```

and run next commands:

```
$ sudo udevadm control --reload
$ sudo udevadm trigger
```

3. Make the minicom work without root:

```
$ sudo usermod -a -G dialout $USER
```

then logout/login in your OS.

4. Install packages required for building:

```
$ sudo apt install git ccache minicom make libncurses5-dev libssl-dev bc bison flex mtools device-tree-compiler
$ pip install pycrypto
```

5. Create directory for all repositories:

```
$ mkdir ~/repos
```

We will use this directory to keep the source code for all software. If you'd like to use another location, please replace `~/repos` with desired directory further.

## Hardware Connections

1. Connect LCD panel and touchscreen to the internal board:

lcd internal connections.jpg

2. Connect LCD board to main board:

lcd-connection.jpg

3. Remove shown jumper, so that OTG port can work in "device" mode:

jumper.jpg

4. Connect OTG USB and UART USB to your machine:

usb-connection.jpg

## Obtaining and Building the Software

### ATF

Obtain ATF:

```
$ cd ~/repos
$ git clone git://git.ti.com/atf/arm-trusted-firmware.git
$ cd arm-trusted-firmware
$ git checkout ti2018.05
```

Configure shell environment for cross-compiling:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu/bin:$PATH
$ export CROSS_COMPILE="ccache aarch64-linux-gnu-"
$ export ARCH=aarch64
```

Build ATF:

```
$ make PLAT=k3 TARGET_BOARD=generic SPD=opteed
```

### OP-TEE

Obtain OP-TEE:

```
$ cd ~/repos
$ git clone git://git.ti.com/optee/ti-optee-os.git
$ cd ti-optee-os
$ git checkout ti2018.05
```

Configure shell environment:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu/bin:$PATH
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi/bin:$PATH
$ export CROSS_COMPILE64="ccache aarch64-linux-gnu-"
$ export CROSS_COMPILE32="ccache arm-eabi-"
$ unset ARCH
```

Build OP-TEE:

```
$ make PLATFORM=k3-am65x CFG_ARM64_core=y
```

### SYSFW

Obtain SysFW:

```
$ cd ~/repos
$ git clone git://git.ti.com/processor-firmware/system-firmware-image-gen.git
$ cd system-firmware-image-gen
$ git checkout 3cbf4003060ead508653e2ec8f25c59bbe813e74
```

Configure shell environment:

```
$ export CROSS_COMPILE="ccache arm-eabi-"
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi/bin:$PATH
$ export ARCH=arm
```

Build SysFW:

```
$ make
```

### U-BOOT

Obtain U-Boot:

```
$ cd ~/repos
$ git clone git://git.ti.com/ti-u-boot/ti-u-boot.git
$ cd ti-u-boot
$ git checkout ti-u-boot-2018.01
```

Apply temp patch from here: https://pastebin.ubuntu.com/p/ZR237HCFKN/

Configure shell environment for Cortex-R5F build:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_arm-eabi/bin:$PATH
$ export CROSS_COMPILE="ccache arm-eabi-"
$ export ARCH=arm
$ unset ATF
$ unset TEE
```

Build U-Boot for Cortex-RF5 core:

```
$ make O=/tmp/r5 distclean
$ make O=/tmp/r5 am65x_evm_r5_defconfig
$ make O=/tmp/r5 -j4
```

Configure shell environment for Cortex-A53 build:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu/bin:$PATH
$ export CROSS_COMPILE="ccache aarch64-linux-gnu-"
$ export ARCH=arm
$ export ATF=~/repos/ti-atf/build/k3/generic/release/bl31.bin
$ export TEE=~/repos/ti-optee-os/out/arm-plat-k3/core/tee-pager.bin
```

Build U-Boot for Cortex-A53 core:

```
$ make O=/tmp/a53 distclean
$ make O=/tmp/a53 am65x_evm_a53_defconfig
$ make O=/tmp/a53 -j4
```

### KERNEL

Obtain Android kernel:

```
$ cd ~/repos
$ git clone git://git.ti.com/ti-linux-kernel/ti-linux-kernel.git
$ cd ti-linux-kernel
$ git checkout ti-android-linux-4.14.y
```

Configure shell environment:

```
$ export PATH=/opt/gcc-linaro-7.3.1-2018.05-x86_64_aarch64-linux-gnu/bin:$PATH
$ export CROSS_COMPILE="ccache aarch64-linux-gnu-"
$ export ARCH=arm64
```

Configure and build the kernel:

```
$ ./ti_config_fragments/defconfig_builder.sh -t ti_sdk_arm64_android_release
$ make ti_sdk_arm64_android_release_defconfig
$ make -j4 Image dtbs modules
```

### AFS

Obtain Android-O MR1 sources:

```
$ cd ~/repos
$ mkdir ti-android-o-mr1
$ cd ti-android-o-mr1
$ repo init -u git://git.ti.com/android/manifest.git -b oreo-mr1-core-dev
$ repo sync -j20
```

Configure and build Android:

```
$ export KERNELDIR=~/repos/ti-linux-kernel
$ . ./build/envsetup.sh
$ lunch am65xevm_full-userdebug
$ make -j4
```

## Flashing and Booting

### UART BOOT: RECOVERY OR FIRST TIME FLASH AND BOOT

Lets Assume `/dev/ttyUSB1` is the first instance and `/dev/ttyUSB2` is second instance.

Configure SYSBOOT DIP-switch for booting from UART:

```
SW2[0:3] = 0010
SW3[0:3] = 1010
SW4[0:1] = 00
```

Script for flashing images via UART:

Toggle line numbers

```
#!/bin/bash

# Run stty to set the baud rate
stty -F /dev/ttyUSB1 115200
stty -F /dev/ttyUSB2 115200

# Send tiboot3.bin  as xmodem, 2 minute timeout
/usr/bin/timeout 120 /usr/bin/sx -k --xmodem tiboot3.bin < /dev/ttyUSB2 > /dev/ttyUSB2

# If we timeout or don't return cleanly (transfer failed), return 1
if [ $? -ne 0 ]; then
    exit 1
fi

# Send sysfw as ymodem, 4 minute timeout.
/usr/bin/timeout 240 /usr/bin/sb -kb --ymodem sysfw.itb < /dev/ttyUSB2 > /dev/ttyUSB2

# If we timeout or don't return cleanly (transfer failed), return 1
if [ $? -ne 0 ]; then
    exit 1
fi

# Send tispl as ymodem, 4 minute timeout
/usr/bin/timeout 240 /usr/bin/sb -kb --ymodem tispl.bin < /dev/ttyUSB1 > /dev/ttyUSB1

# If we timeout or don't return cleanly (transfer failed), return 1
if [ $? -ne 0 ]; then
    exit 1
fi

# Send U-Boot as ymodem, 4 minute timeout
/usr/bin/timeout 240 /usr/bin/sb -kb --ymodem u-boot.img < /dev/ttyUSB1 > /dev/ttyUSB1

# If we timeout or don't return cleanly (transfer failed), return 1
if [ $? -ne 0 ]; then
    exit 1
fi

sleep 1
# Send an echo to be sure that we will break into autoboot
echo > /dev/ttyUSB1

mkdir /tmp/images
cd /tmp/images
cp /tmp/a53/tispl.bin .
cp /tmp/a53/u-boot.img .
cp ~/repos/system-firmware-image-gen/sysfw.itb .
dd if=/dev/zero of=bootloader.img bs=1M count=8
/sbin/mkfs.vfat bootloader.img
mcopy -i bootloader.img tispl.bin ::tispl.bin
mcopy -i bootloader.img u-boot.img ::u-boot.img
mcopy -i bootloader.img sysfw.itb ::sysfw.itb

cp /tmp/r5/tiboot3.bin .
cp ~/repos/ti-android-o-mr1/out/target/product/am65xevm/*.img .

# Flash bootloader in eMMC

echo 'env default -f -a' > /dev/ttyUSB1
echo 'setenv partitions $partitions_android' > /dev/ttyUSB1
echo 'env save' > /dev/ttyUSB1

echo 'fas 0' > /dev/ttyUSB1
fastboot oem format
fastboot flash tiboot3 tiboot3.bin
fastboot flash bootloader bootloader.img
```

### SD BOOT: RECOVERY OR FIRST TIME FLASH AND BOOT

Let's assume your SD card dev file is `/dev/mmcblk0`.

Format SD card:

```
$ sudo dd if=/dev/zero of=/dev/mmcblk0 bs=1M count=1
$ sudo sfdisk /dev/mmcblk0 << EOF
2048,1G,0x0c,*
,,L,-
EOF

$ sudo mkfs.vfat -F 32 -n "boot" /dev/mmcblk0p1
$ sudo mkfs.ext4 -F -L "rootfs" /dev/mmcblk0p2
```

Copy files to SD card:

```
sudo mkdir /mnt/boot
sudo mount /dev/mmcblk0p1 /mnt/boot
sudo cp ~/repos/system-firmware-image-gen/sysfw.itb /mnt/boot
sudo cp /tmp/r5/tiboot3.bin /mnt/boot
sudo cp /tmp/a53/tispl.bin /mnt/boot
sudo cp /tmp/a53/u-boot.img /mnt/boot
sudo sync
sudo umount /mnt/boot
```

Configure SYSBOOT DIP-switch for booting from SD card:

```
SW2[0:3] = 0010
SW3[0:3] = 0110
SW4[0:1] = 11
```

Insert prepared SD card into the slot. Power on the board. Press SPACE to get into U-Boot shell.

### EMMC BOOT: PRODUCTION FLASH AND BOOT

You can use either SD or UART Boot to Recover the device or Boot for the first time.

Configure SYSBOOT DIP-switch for booting from eMMC (user area mode):

```
SW2[0:3] = 0000
SW3[0:3] = 0110
SW4[0:1] = 00
```

Prepare the U-Boot environment:

```
=> env default -f -a
=> setenv partitions $partitions_android
=> env save
=> fastboot 0
```

Flash images to eMMC:

```
$ fastboot oem format
$ fastboot flash tiboot3 tiboot3.bin
$ fastboot flash bootloader bootloader.img
$ fastboot flash vendor vendor.img
$ fastboot flash userdata userdata.img
$ fastboot flash boot boot_fit.img
$ fastboot flash system system.img
```

Reboot the board:

```
$ fastboot reboot
```

You should see kernel boot log in minicom and Android boot animation/lock screen on LCD panel.

If you have Automation Board (optional), you can use it for switching between UART and eMMC boot mode. More at https://wiki.linaro.org/Boards/TI_Automation .

## Bringing up Ethernet on Android

Internet connection can be setup using next commands:

```
$ su
# ip addr add 192.168.0.100 dev eth0
# ip link set eth0 up
# ip route add 192.168.0.0/24 dev eth0
# ip route add default via 192.168.0.1 dev eth0
# ip rule add from all lookup main pref 99

# ip addr show dev eth0 scope global
# ip route show
# ping 192.168.0.1

# ndc network create 100
# ndc network interface add 100 eth0
# ndc resolver setnetdns 100 localdomain 8.8.8.8 8.8.4.4
# ndc network default set 100

# ping google.com
```

If the board is connected via Ethernet to your PC/laptop (not into router), please enable IP forwarding and setup iptables correspondingly:

1. Edit /etc/sysctl.conf and add/change next line:

```
net.ipv4.ip_forward=1
```

and do next to enable this setting for current session:

```
$ sudo service procps start
```

2. Configure IPtables to grant access for your board's subnet to public network (with Internet):

```
$ sudo iptables -A POSTROUTING -t nat -o wlan0 -s 192.168.0.0/24 -j MASQUERADE
```

where public interface is `wlan0` and `192.168.0.0/24` is Ethernet network.

Details: https://wiki.linaro.org/LMG/Kernel/PPP#Bring_up_Ethernet

# How to build the LCR AM65X Pie build

There is a linaro-build.sh script that we could use to build images for LCR AM65X Pie build.

Here are the instructions on that.

## get the linaro-build.sh script

```
$ mkdir am65x-workspace && cd am65x-workspace
$ wget https://android-git.linaro.org/android-build-configs.git/plain/linaro-build.sh -O  linaro-build.sh
$ chmod +x linaro-build.sh
```

## download all the sources and build images

```
$ ./linaro-build.sh -c lcr-reference-am65x-p
```

It will help to download the necessary source files, including AOSP source, kernel source, firmware source, etc, and build the images for AM65X platforms automatically(might need to input password for the use of sudo in the script) according to the configurations in lcr-reference-am65x-p file here:

https://android-git.linaro.org/android-build-configs.git/tree/

If you have local AOSP mirror, you could run as following to repo sync the AOSP source from local mirror quickly:

```
$ ./linaro-build.sh -c lcr-reference-am65x-p -m <local-mirror-url>
```

please run

```
$ ./linaro-build.sh -h
```

to check the all available options.

If there is any building failure, please check or share the build-am65xevm_full.log file for investigation.

After the script finished successfully, the following files would be generated under out/target/product/am65xevm:

- tiboot3.bin
- tispl.bin
- sysfw.itb
- u-boot.img
- bootloader.img
- boot_fit.img
- system.img
- userdata.img
- vendor.img

### Prepare the sd card for boot

***Please make sure the board is powered off first, then remove the sdcard, otherwise the data in sdcard might be damaged.*** please partition and format the sd card with following commands:

```
$ sudo dd if=/dev/zero of=/dev/mmcblk0 bs=1M count=1
$ sudo sfdisk /dev/mmcblk0 << EOF
2048,1G,0x0c,*
,,L,-
EOF

$ sudo mkfs.vfat -F 32 -n "boot" /dev/mmcblk0p1
$ sudo mkfs.ext4 -F -L "rootfs" /dev/mmcblk0p2
```

and copy the newly generated firmware files to sdcard boot partition, like via command following:

```
$ sudo mkdir -p /mnt/boot
$ sudo mount /dev/mmcblk0p1 /mnt/boot
$ sudo cp out/target/product/am65xevm/{sysfw.itb,tiboot3.bin,tispl.bin,u-boot.img} /mnt/boot
$ sudo sync
$ sudo umount /mnt/boot
```

### Configure SYSBOOT DIP-switch for booting from SDCard

***Please make sure the board is powered off first, then change the switches***

```
SW2[0:3] = 0000
SW3[0:3] = 0110
```

### Boot to fastboot mode with the SDCard

***Please make sure the board is powered off first, then insert the sdcard***

Power on the board after sdcard inserted, step into the u-boot shell, and run following u-boot commands:

```
=> env default -f -a
=> setenv partitions $partitions_android
=> env save
=> fastboot 0
```

### Flashing the firmware images, and android images

Flash all the firmware and android images to eMMC from host side.

```
$ fastboot oem format
$ fastboot flash tiboot3 tiboot3.bin
$ fastboot flash bootloader bootloader.img
$ fastboot flash boot boot_fit.img
$ fastboot flash system system.img
$ fastboot flash userdata userdata.img  # would take about 6-10 minutes to finish.
$ fastboot flash vendor vendor.img
```

### Configure SYSBOOT DIP-switch for booting from eMMC

***Please make sure the board is powered off first, then change the switches and remove the sdcard***

Please do like this:

1. Power off the board.
2. Remove the sdcard,
3. Config the SYSBOOT DIP-switch like following:

```
SW2[0:3] = 0000
SW3[0:3] = 0110
SW4[0:1] = 00
```

### Boot Android Images

Power on the board, check the output on the serial console, adb connection via adb devices, and see if it could boot to the home screen on the LCD panel.

If everything works well, then you would see the device listed in the output of "adb devices" if all the otg-usb cable connected correctly.

## References

[1] http://git.ti.com/cgit/cgit.cgi/ti-u-boot/ti-u-boot.git/tree/board/ti/am65x/README?h=ti-u-boot-2018.01

*AOSP/Boards/AM65xEVM (last modified 2019-06-11 18:44:17)*

Portions of this content © 2010 - 2019 **Linaro**

Sitemap | Legal | Contact | 深圳市秋家源贸易有限公司 粤ICP备17104008号-2

MoinMoin Powered Python Powered GPL licensed